

UNFINISHED

Understanding LED Matrix and Microcontroller Specs:

Before we begin, it is important to acknowledge a huge obstacle in connecting the Teensy Digital I/O pins. Unfortunately, the data input pins from the LED matrix run on 5V, and the Teensy only runs logic at 3.3V. There are a couple of options I considered, including buying a specialized level shifter, making my own level shifter, and even using an Arduino UNO, since its logic can run at 5V. The easiest option would've been to use the Arduino UNO, but there are many reasons why this is a bad idea.

First off, an LED matrix is a whole different beast compared to an I2C or SPI LED. Those types of LEDs have their own memory, so it only takes a single command to keep a single LED on since it can refresh its data. On the other hand, an LED matrix is barebones and has no memory. This means that it doesn't take just a single command to keep on one LED. In fact, LED matrices use something called scan drivers, which only turn on a fraction of the LED rows (2 rows at a time for a 32x64 matrix) at a time. In order to appear like the LED always on, the driver must be constantly updated to pulse LEDs, which requires lots of processing power and RAM, both of which are lacking on the Arduino UNO. For comparison, the Arduino has 16mhz of processing power and 2kb of RAM, while the Teensy 4.1 has 600mhz and 1mb of RAM.

Now that we know why the Teensy is a much better option, it's time to address how to convert Teensy's logic pins from 3.3V to 5V. At first I thought about using an op-amp to gain voltage for each data input pin, but this seemed both inefficient and not their nature since op-amps aren't meant for sharp edge data like digital logic. There is a component online which makes the wiring very easy called a SmartMatrix SmartLED Shield, but I decided to create the circuit myself to reduce costs.

- 3.3V to 5V Level Shifter Support Circuit
 - Plan: From Teensy pins, we will use R1, B1, G1, ..., OE, CLK, LAT, all which output a HIGH 3.3V signal. The LED matrix only supports 5V logic, so with the use of an Octal Bus Transceiver, we can easily fix this issue. However, we still need to synchronize the extremely fast LED pulses from the matrix. We will program the matrix to work a certain way through Arduino IDE, and teensy will automatically use a 5 bit number (A-E addresses on the LED Matrix) to locate the row position it needs to change. For example, if my Teensy CPU selects the bit word 00010, it will augment only row 2 at a single point in time.
 - The LED matrix has a 1/16th driver scan, so it will change two rows at a time.
 - SN74HCT245N chip is an Octal Bus Transceiver with 3-State Outputs. In short, this chip will receive a 3.3V input and set HIGH signal equal to 5V, perfect for our problem. The LED matrix requires 13 parallel signals, so we will need two of these chips.

- SN74HCT374N - Technically, the D Flip Flop isn't needed, but the software would have trouble timing with the LAT input from the LED matrix, so its possible glitches would occur. Using a D Flip Flop will synchronize A-E inputs with the A-E address inputs

I wanted to make this project mobile, like a mini game portable game console like the gameboy advance, or nintendo switch.

For finalization purposes, I want to turn this LED matrix into a handheld console similar to a Gameboy Advance. I will customize my own PCB using Altium Designer, and design and 3D print the shell using tinkercad. I also plan to add audio in the future using a low voltage amplifier like a LM386 low voltage amplifier and a small 8ohm speaker.

- Because the LED matrix has about a 3A maximum current, it is likely that in a poorly ventilated shell that the components will overheat. To solve this, we will add ventilation slits so that the heat escapes. Also, we will separate the power input from the circuitry, but this will be addressed when designing the PCB.

Schematics:

SN74AHCT254N:

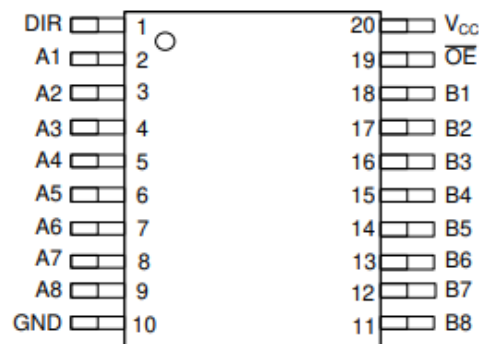
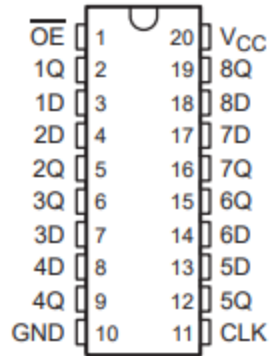


Figure 5-1. SN54AHCT245: J or W SN74AHCT245: DB, DGV, DW, N, NS, PW or DGS Package, 20-Pin CDIP, CFP, SSOP, TVSOP, SOIC, PDIP, SOP, TSSOP, or VSSOP (Top View)

SN74HCT374N pinout:

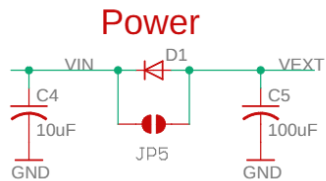
SN54HCT374 . . . J OR W PACKAGE
 SN74HCT374 . . . DB, DW, N, NS, OR PW PACKAGE
 (TOP VIEW)



LED SmartShield:

Teensy:

Power:

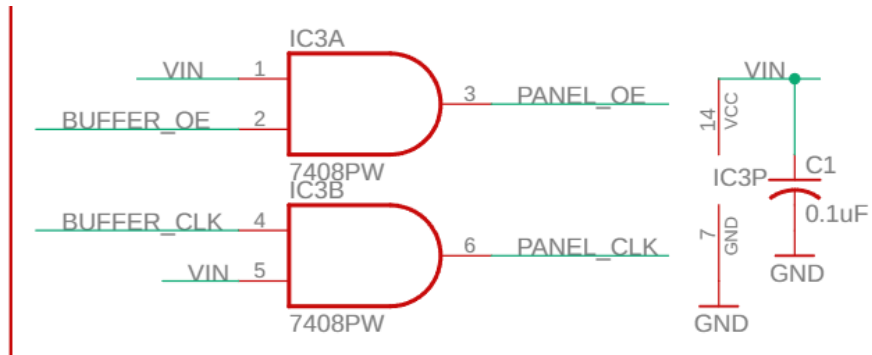
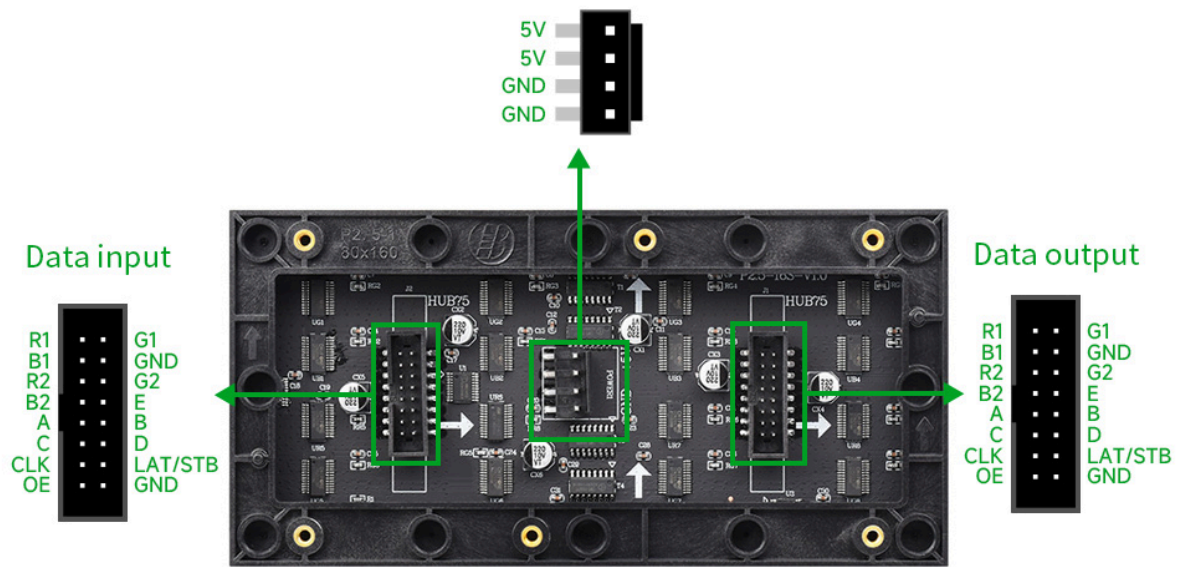


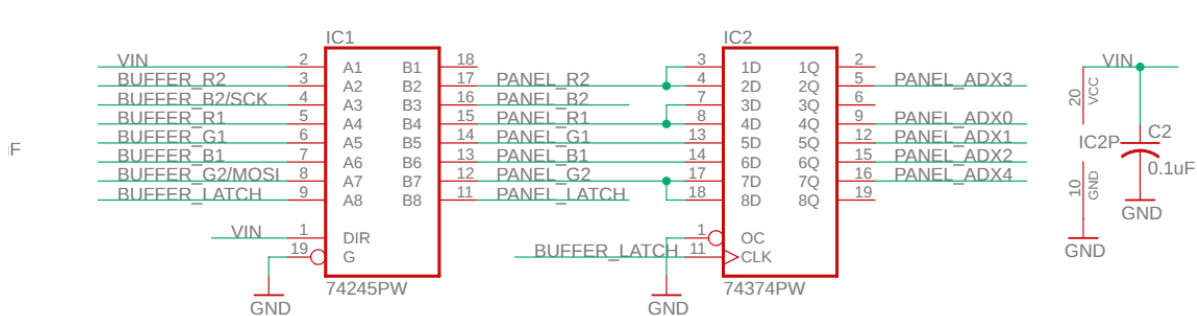
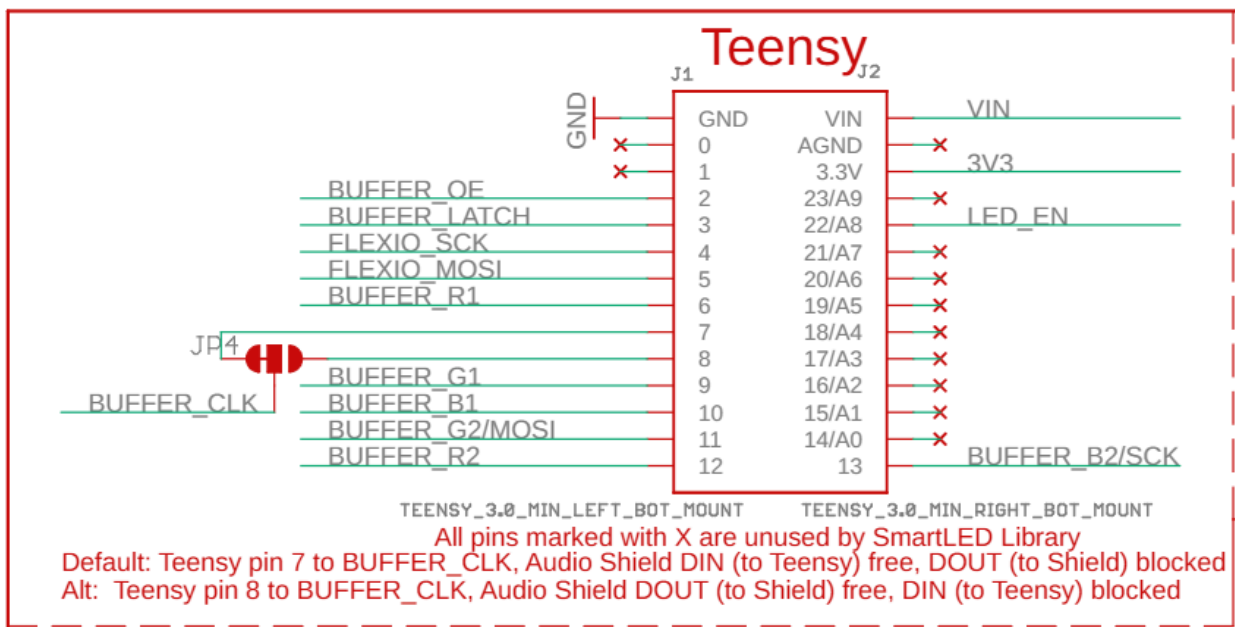
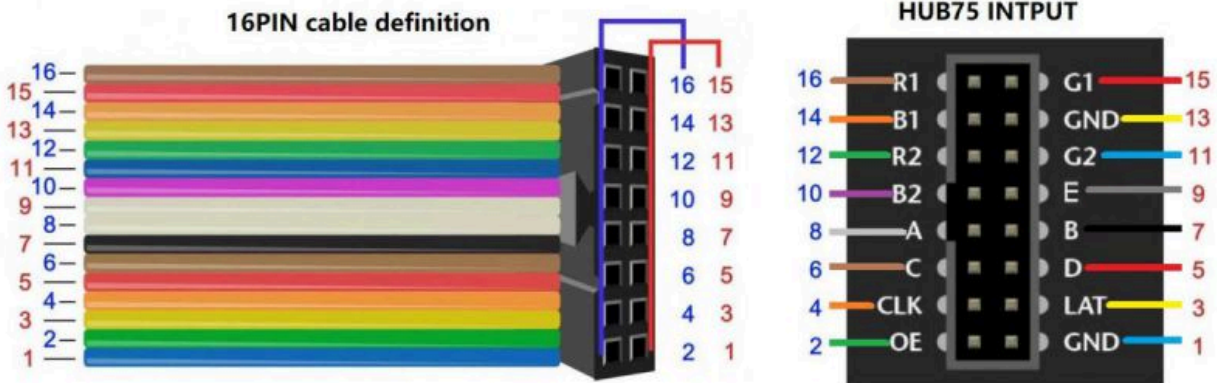
LEDs are powered externally, Teensy can be powered from VEXT or from USB
 Alt: Short JP5, drive up to 2A from USB to VEXT

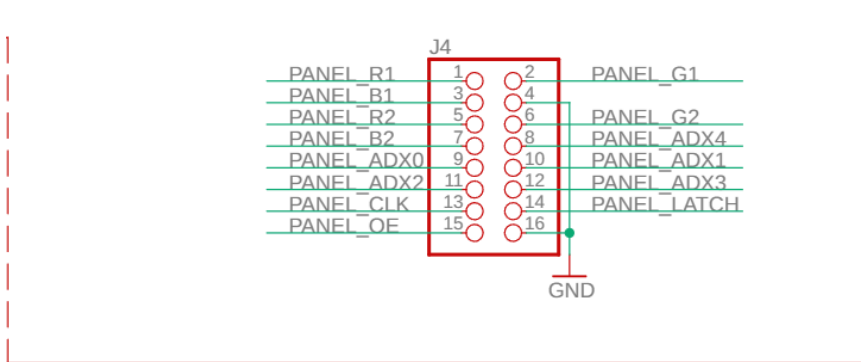
C5 is optional (not placed), it can be added to lessen the effects of kickback when switching long strings of LEDs on and off using USB power

Data Input:

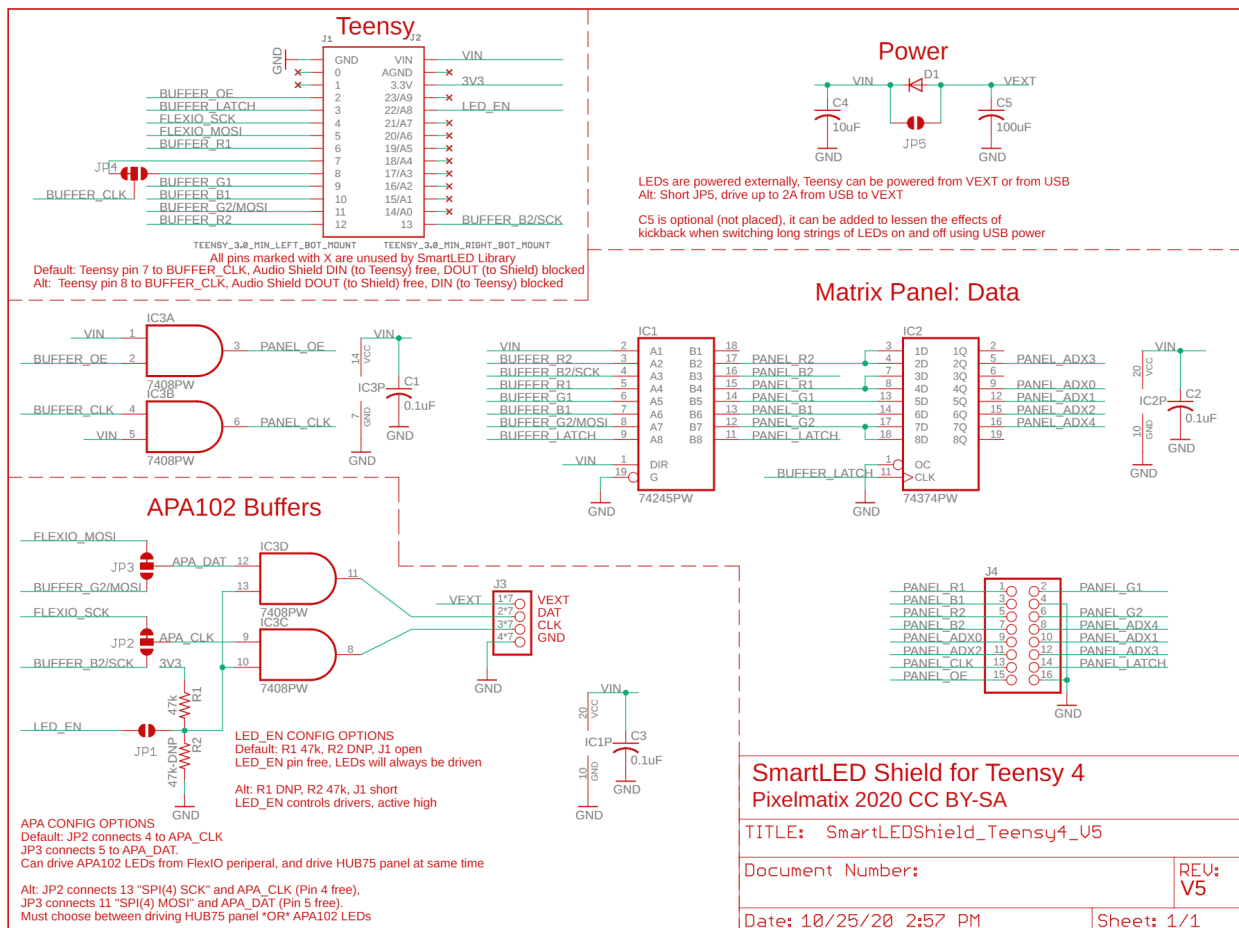
- IC1 is the octal bus transceiver
- IC2 is the D Flip Flop
- J4 is the LED connector pins



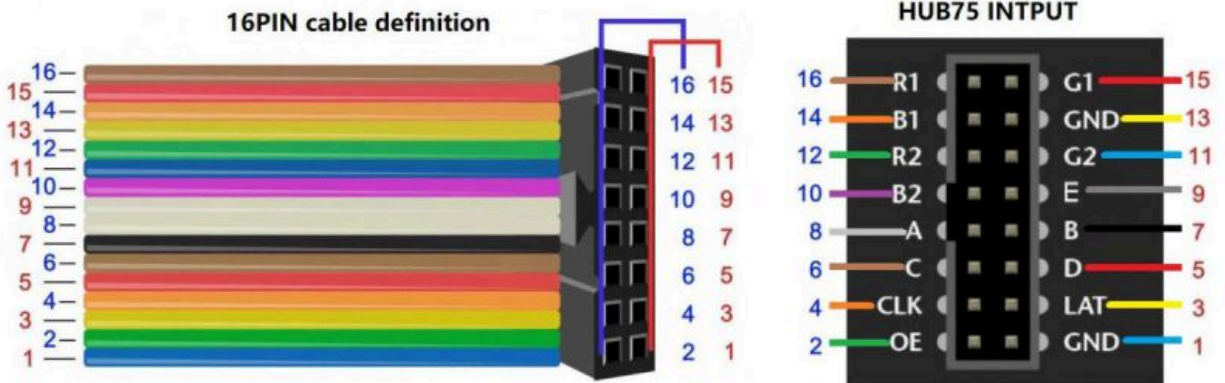




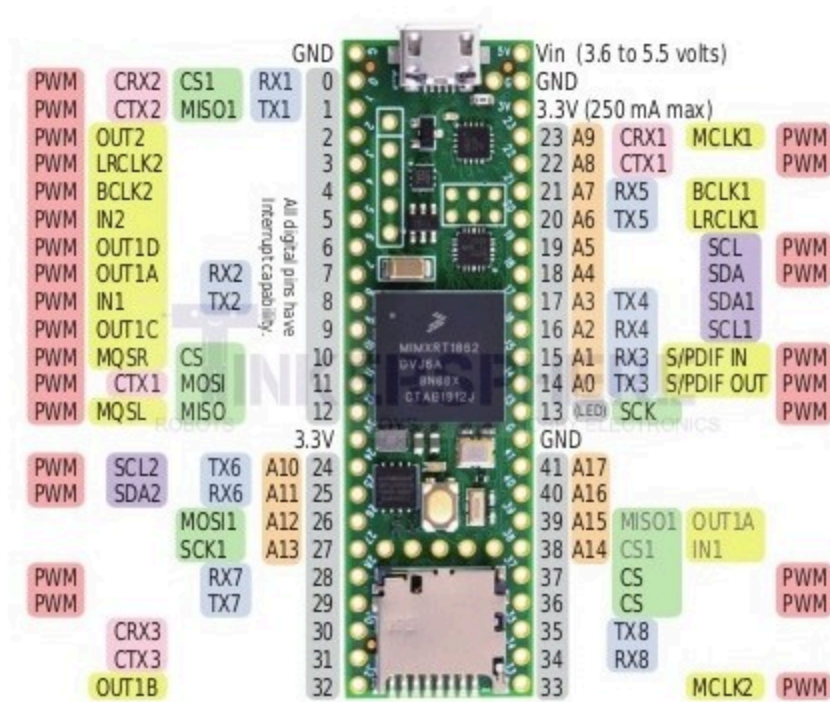
Full SmartShield Schematic:



Pinout for 32x64 LED matrix(Ignore Data Output):



PIN	DESCRIPTION	PIN	DESCRIPTION
+5V	5V power input	GND	Ground
R1	R higher bit data	R2	R lower bit data
G1	G higher bit data	G2	G lower bit data
B1	B higher bit data	B2	B lower bit data
A	A line selection	B	B line selection
C	C line selection	D	D line selection
E	E line selection	CLK	clock input
LAT/STB	latch pin	OE	output enable



References:

- [SmartMatrix/extras/hardware/Teensy4 at master · pixelmatix/SmartMatrix · GitHub](#)
- <https://www.ti.com/lit/ds/symlink/sn74ahct245.pdf?ts=1759510386805>
- [RGB full-color LED matrix panel, 2.5mm Pitch, 64x32 pixels, adjustable brightness | RGB-Matrix-P2.5-64x32](#)